

ATIVIDADE – ANÁLISE DE QUALIDADE DE CÓDIGO COM ESLINT

Disciplina

Qualidade de Software

Objetivo

Compreender o papel da análise estática de código no processo de garantia da qualidade de software, utilizando a ferramenta ESLint para identificar, analisar e corrigir problemas em aplicações desenvolvidas em JavaScript ou TypeScript.

Contexto

Durante o desenvolvimento de software, diversos problemas podem surgir antes mesmo da execução do sistema, como:

- erros de programação;
- inconsistências de estilo;
- más práticas de desenvolvimento;
- código de difícil manutenção;
- vulnerabilidades causadas por descuidos de implementação.

Ferramentas de análise estática ajudam a detectar esses problemas automaticamente, aumentando a qualidade do produto e reduzindo custos de manutenção.

Uma das ferramentas mais utilizadas para esse fim no ecossistema JavaScript e TypeScript é o ESLint.

Nesta atividade você deverá utilizar o ESLint em um projeto próprio, analisar os resultados obtidos e produzir evidências do processo de melhoria do código.

Parte 1 – Desenvolvimento da Aplicação

Desenvolva uma aplicação própria utilizando:

- Node.js + JavaScript

ou

- Node.js + TypeScript

A aplicação deve possuir complexidade suficiente para permitir uma análise significativa pelo ESLint.

Requisitos mínimos:

- mínimo de 5 arquivos fonte;
- mínimo de 300 linhas de código;
- utilização de funções;
- utilização de estruturas condicionais;
- utilização de estruturas de repetição;
- utilização de módulos/importações;
- interação com entrada de dados ou arquivos.

O tema da aplicação é livre.

Exemplos:

- sistema de biblioteca;
- controle financeiro;
- agenda de contatos;
- gerenciamento de estoque;
- sistema de pedidos;
- cadastro de alunos;
- sistema de reservas.

Parte 2 – Estudo da Ferramenta

Elabore uma seção teórica no relatório explicando:

1. O que é o ESLint

Apresente:

- definição;
- objetivo;
- histórico da ferramenta;
- contexto de utilização.

2. O que é Análise Estática

Explique:

- conceito;
- diferenças entre análise estática e testes;
- vantagens;
- limitações.

3. Aplicações do ESLint

Descreva como a ferramenta auxilia em:

- padronização de código;
- identificação de erros;
- manutenção;
- qualidade;
- trabalho em equipe;
- integração contínua.

Parte 3 – Instalação e Configuração

Documente todo o processo realizado.

Apresente:

Instalação

Comandos utilizados.

Exemplo:

```
npm install eslint --save-dev
```

Inicialização

Comandos utilizados.

Exemplo:

```
npx eslint --init
```

Configuração

Explique:

- regras escolhidas;
- estilo adotado;
- principais opções configuradas.

Inclua capturas de tela.

Parte 4 – Execução da Ferramenta

Execute o ESLint no projeto.

Exemplo:

`npx eslint .`

Apresente:

- quantidade de erros encontrados;
- quantidade de avisos encontrados;
- principais categorias de problemas identificados.

Inclua capturas de tela da execução.

Parte 5 – Correção Automática

Execute:

`npx eslint . --fix`

Documente:

- quais problemas foram corrigidos automaticamente;
- quais problemas precisaram de intervenção manual;
- limitações da correção automática.

Inclua evidências.

Parte 6 – Comparação Antes e Depois

Selecione pelo menos 10 exemplos de correções realizadas.

Para cada exemplo apresente:

Antes

Trecho original.

Depois

Trecho corrigido.

Explicação

Descreva:

- qual era o problema;
- qual regra foi aplicada;
- qual o benefício da correção.

Parte 7 – Avaliação dos Resultados

Analise criticamente:

O ESLint encontrou problemas relevantes?

Os problemas poderiam causar falhas reais?

A ferramenta ajudou a melhorar a legibilidade?

A ferramenta ajudou a padronizar o código?

Quais limitações foram observadas?

Você utilizaria essa ferramenta em projetos reais?

Justifique suas respostas.

Entregáveis

Entregar:

1. Projeto

Código-fonte completo antes e depois de usar a ferramenta.

2. Relatório

Formato PDF.

Mínimo:

- 8 páginas
- máximo livre

3. Evidências

Capturas de tela contendo:

- instalação;
- configuração;
- execução;
- correções;
- resultados finais.